Bazele Inteligenţei Artificiale

Prof. Dr. Călin-Adrian POPA

Cursul 5

28 Octombrie 2025

5 Clustering

- *clustering* is the process of grouping similar objects together into *clusters*
- there are two kinds of inputs we might use
  - in *feature-based clustering*, the input to the algorithm is an $n \times d$ feature matrix or design matrix $\boldsymbol{X}$. Feature-based clustering has the advantage that it is applicable to “raw”, potentially noisy data.
  - in *similarity-based clustering*, the input to the algorithm is an $n \times n$ dissimilarity matrix or distance matrix $\boldsymbol{D}$. Similarity-based clustering has the advantage that it allows for easy inclusion of domain-specific similarity or kernel functions.

5 Clustering

- in clustering, just like in classification, each instance gets assigned to a group
- however, unlike classification, clustering is an *unsupervised* task
- consider Figure 1: on the left is the `iris` dataset, where each instance's species (i.e., its class) is represented with a different marker

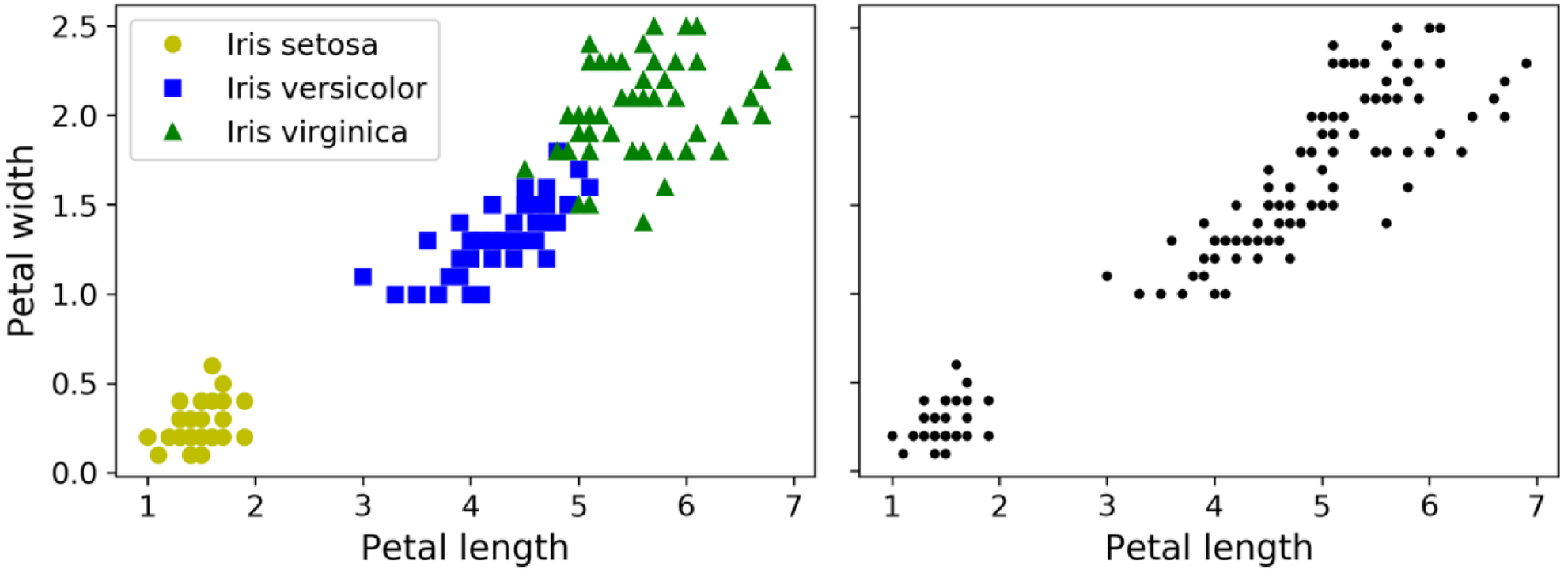


Figure 1: Classification (left) versus clustering (right).

- it is a labeled dataset, for which classification algorithms such as logistic regression, SVMs, or random forest classifiers are well suited
- on the right is the same dataset, but without the labels, so we cannot use a classification algorithm anymore
- this is where clustering algorithms step in: many of them can easily detect the lower-left cluster
- it is also quite easy to see with our own eyes, but it is not so obvious that the upper-right cluster is composed of two distinct sub-clusters
- that said, the dataset has two additional features (sepal length and sepal width), not represented here, and clustering algorithms can make good use of all features, so in fact they identify the three clusters fairly well

5 Clustering

- there is no universal definition of what a cluster is: it really depends on the context, and different algorithms will capture different kinds of clusters
- some algorithms look for instances centered around a particular point, called a *centroid*
- others look for continuous regions of densely packed instances: these clusters can take on any shape
- some algorithms are hierarchical, looking for clusters of clusters, and the list goes on

5.1 Evaluating clustering methods

- clustering is an unsupervised learning technique, so it is hard to *evaluate* the quality of the output of any given method
- intuitively, the goal of clustering is to assign points that are similar to the same cluster, and to ensure that points that are dissimilar are in different clusters
- there are several ways of measuring these quantities, but internal criteria may be of limited use
- an alternative is to rely on some external form of data with which to validate the method
- for example, if we have labels for each object, then we can assume that objects with the same label are similar
- we can then use the metrics we discuss below to quantify the quality of the clusters
- if we do not have labels, but we have a reference clustering, we can derive labels from that clustering

5.1 Evaluating clustering methods

- let n_{ij} be the number of objects in cluster i that belong to class j, and let $n_i = \sum_{j=1}^{C} n_{ij}$ be the total number of objects in cluster i
- define $p_{ij} = n_{ij}/n_i$; this is the empirical distribution over class labels for cluster i
- we define the *purity* of a cluster as $p_i = \max_j p_{ij}$, and the overall purity of a clustering as:

$$\text{purity} = \sum_i \frac{n_i}{n} p_i.$$

- the purity ranges between 0 (bad) and 1 (good); however, we can trivially achieve a purity of 1 by putting each object into its own cluster, so this measure does not penalize for the number of clusters

- for example, in Figure 2, we have that the purity is:

$$\frac{6}{17}\frac{5}{6} + \frac{6}{17}\frac{4}{6} + \frac{5}{17}\frac{3}{5} = \frac{5+4+3}{17} = 0.71.$$

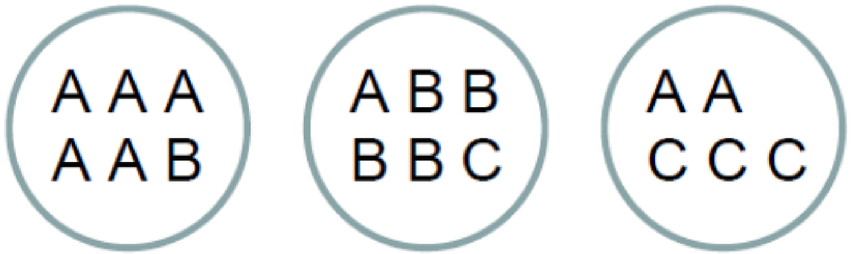


Figure 2: Three clusters with labeled objects inside.

5.1 Evaluating clustering methods

- let $U = \{u_1, \ldots, u_R\}$ and $V = \{v_1, \ldots, v_C\}$ be two different partitions of the n data points
- for example, U might be the estimated clustering and V the reference clustering derived from the class labels
- now define a 2×2 contingency table, containing the following numbers:
  - TP is the number of pairs that are in the same cluster in both U and V (true positives)
  - TN is the number of pairs that are in the different clusters in both U and V (true negatives)
  - FN is the number of pairs that are in the different clusters in U but the same cluster in V (false negatives)
  - FP is the number of pairs that are in the same cluster in U but different clusters in V (false positives)
- a common summary statistic is the *Rand index*:

$$R = \frac{TP + TN}{TP + FP + FN + TN}.$$

- this can be interpreted as the fraction of clustering decisions that are correct; clearly $0 \leq R \leq 1$

5.1 Evaluating clustering methods

- for example, consider Figure 2; the three clusters contain 6, 6, and 5 points, so the number of "positives" (i.e., pairs of objects put in the same cluster, regardless of label) is:

$$TP + FP = C_6^2 + C_6^2 + C_5^2 = 40.$$

- of these, the number of true positives is given by:

$$TP = C_5^2 + C_4^2 + C_3^2 + C_2^2 = 20,$$

  where the last two terms come from cluster 3: there are C_3^2 pairs labeled C and C_2^2 pairs labeled A
- so $FP = 40 - 20 = 20$; similarly, one can show that $FN = 24$ and $TN = 72$
- thus, the Rand index is $(20 + 72)/(20 + 20 + 24 + 72) = 0.68$

5.1 Evaluating clustering methods

- the Rand index only achieves its lower bound of 0 if $TP = TN = 0$, which is a rare event
- one can define an *adjusted Rand index* (*ARI*) as follows:

$$ARI = \frac{\text{index} - \text{expected index}}{\text{max index} - \text{expected index}}.$$

- here, the model of randomness is based on using the generalized hyper-geometric distribution, i.e., the two partitions are picked at random subject to having the original number of classes and objects in each, and then the expected value of $TP + TN$ is computed
- this model can be used to compute the statistical significance of the Rand index

5.1 Evaluating clustering methods

- another way to measure cluster quality is to compute the *mutual information* between two candidate partitions U and V
- to do this, let

$$p_{UV}(i,j) = \frac{|u_i \cap v_j|}{n}$$

  be the probability that a randomly chosen object belongs to cluster u_i in U and v_j in V
- also, let $p_U(i) = |u_i|/n$ be the probability that a randomly chosen object belongs to cluster u_i in U; define $p_V(j) = |v_j|/n$ similarly
- then we have:

$$I(U,V) = \sum_{i=1}^{R}\sum_{j=1}^{C} p_{UV}(i,j)\log\frac{p_{UV}(i,j)}{p_U(i)p_V(j)}.$$

5.1 Evaluating clustering methods

- this lies between 0 and $\min\{H(U), H(V)\}$, where $H(\cdot)$ denotes the entropy
- unfortunately, the maximum value can be achieved by using lots of small clusters, which have low entropy
- to compensate for this, we can use the *normalized mutual information* (*NMI*):

$$NMI(U, V) = \frac{I(U, V)}{(H(U) + H(V))/2}.$$

- this lies between 0 and 1

5.2 *K*-means clustering

- consider the *unlabeled* dataset represented in Figure 3: we can clearly see five blobs of instances
- the *K*-means algorithm is a simple algorithm capable of clustering this kind of dataset very quickly and efficiently, often in just a few iterations

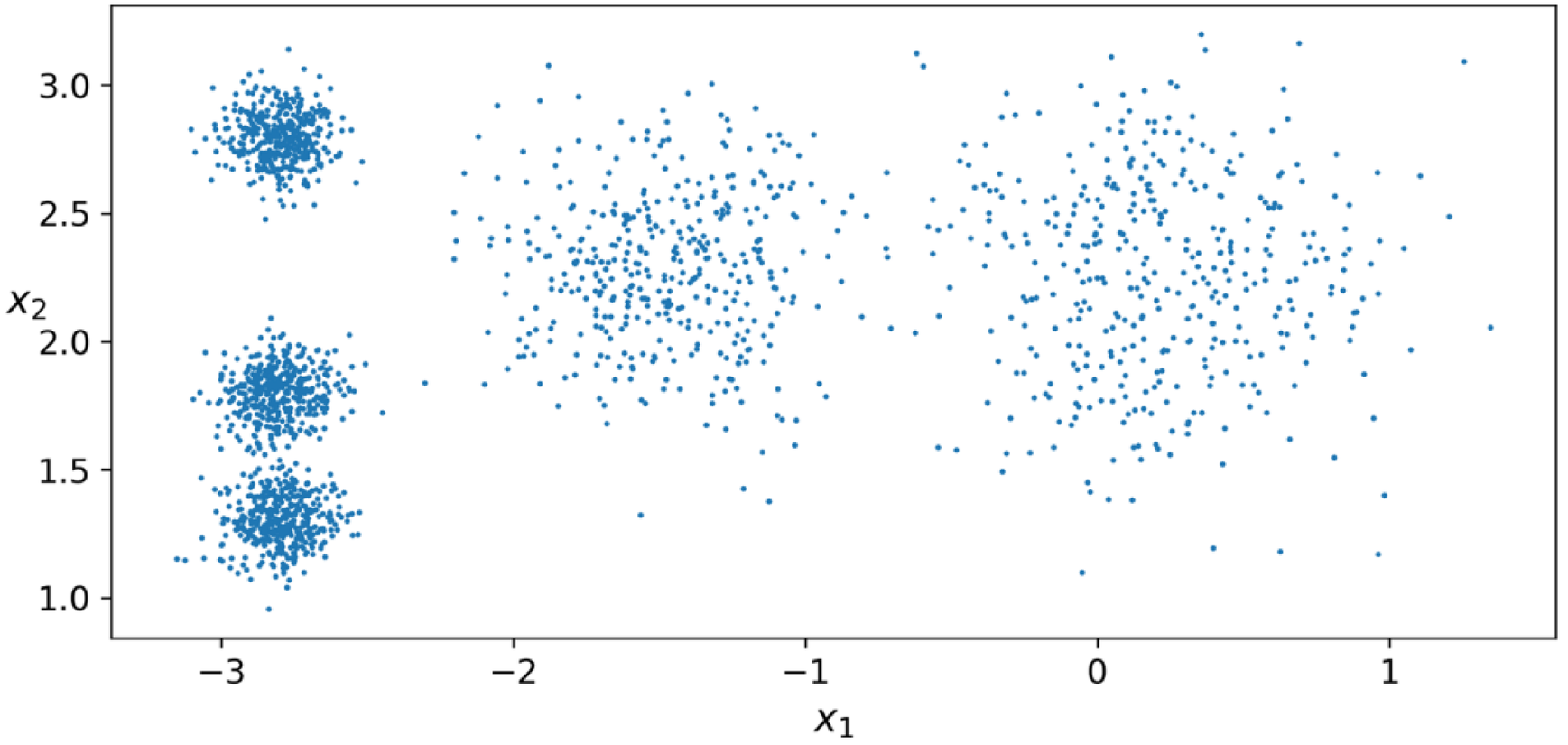


Figure 3: An unlabeled dataset composed of five blobs of instances.

5.2 K-means clustering

- the K-means clusterer applied on this dataset will try to find each blob's center and assign each instance to the closest blob
- note that we have to specify the number of clusters K that the algorithm must find
- in this example, it is pretty obvious from looking at the data that K should be set to 5, but in general it is not that easy; we will discuss this shortly
- each instance was assigned to one of the five clusters
- in the context of clustering, an instance's *label* is the index of the cluster that this instance gets assigned to by the algorithm: this is not to be confused with the class labels in classification (remember that clustering is an unsupervised learning task)
- we can easily assign new instances to the cluster whose centroid is closest

5.2 K-means clustering

- if we plot the clusterer's decision boundaries, we get a *Voronoi tessellation* (see Figure 4, where each centroid is represented with an $\times$)

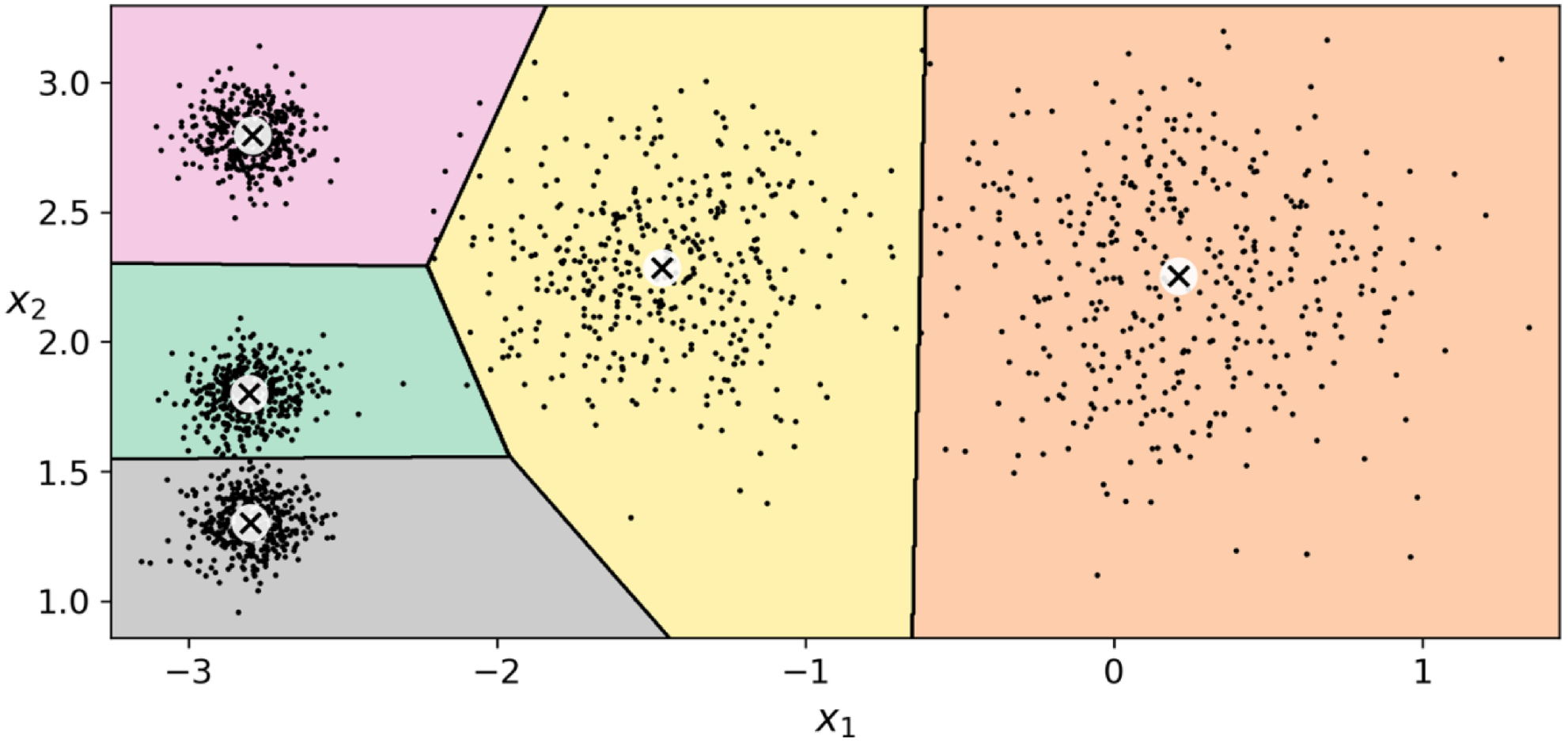


Figure 4: K-means decision boundaries (Voronoi tessellation).

5.2 *K*-means clustering

- the vast majority of the instances were clearly assigned to the appropriate cluster, but a few instances were probably mislabeled (especially near the boundary between the top-left cluster and the central cluster)
- indeed, the *K*-means algorithm does not behave very well when the blobs have very different diameters, because all it cares about when assigning an instance to a cluster is the distance to the centroid
- instead of assigning each instance to a single cluster, which is called *hard clustering*, it can be useful to give each instance a score per cluster, which is called *soft clustering*
- the score can be the distance between the instance and the centroid; conversely, it can be a similarity score (or affinity), such as the Gaussian Radial Basis Function (RBF)

5.2 K-means clustering

- so, how does the algorithm work?
- if we were given the centroids, we could easily label all the instances in the dataset by assigning each of them to the cluster whose centroid is closest
- conversely, if we were given all the instance labels, we could easily locate all the centroids by computing the mean of the instances for each cluster
- but we are given neither the labels nor the centroids, so how can we proceed?
- we just start by placing the centroids randomly (e.g., by picking K instances at random and using their locations as centroids)

5.2 K-means clustering

- then we label the instances, update the centroids, label the instances, update the centroids, and so on, until the centroids stop moving
- the algorithm is guaranteed to converge in a finite number of steps (usually quite small)
- it will not oscillate forever because the mean squared distance between the instances and their closest centroid can only go down at each step
- we can see the algorithm in action in Figure 5: the centroids are initialized randomly (top left), then the instances are labeled (top right), then the centroids are updated (center left), the instances are relabeled (center right), and so on
- as we can see, in just three iterations, the algorithm has reached a clustering that seems close to optimal

5.2 K-means clustering

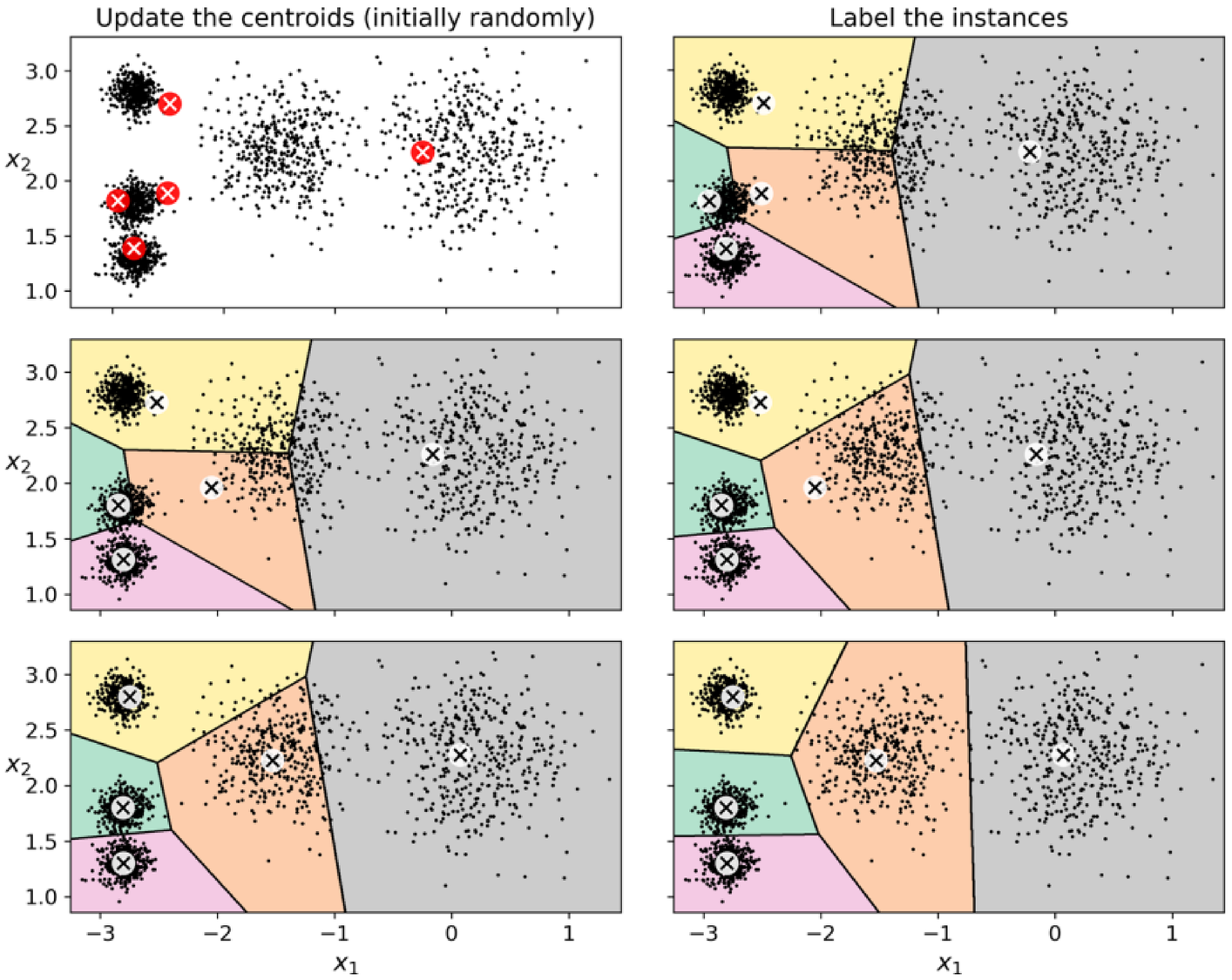


Figure 5: The K-means algorithm.

5.2 K-means clustering

- the computational complexity of the algorithm is generally linear with regard to the number of instances n, the number of clusters K, and the number of dimensions d
- however, this is only true when the data has a clustering structure
- if it does not, then, in the worst-case scenario, the complexity can increase exponentially with the number of instances
- in practice, this rarely happens, and K-means is generally one of the fastest clustering algorithms

5.2 K-means clustering

- although the algorithm is guaranteed to converge, it may not converge to the right solution (i.e., it may converge to a local optimum): whether it does or not depends on the centroid initialization
- Figure 6 shows two suboptimal solutions that the algorithm can converge to if we are unlucky with the random initialization step

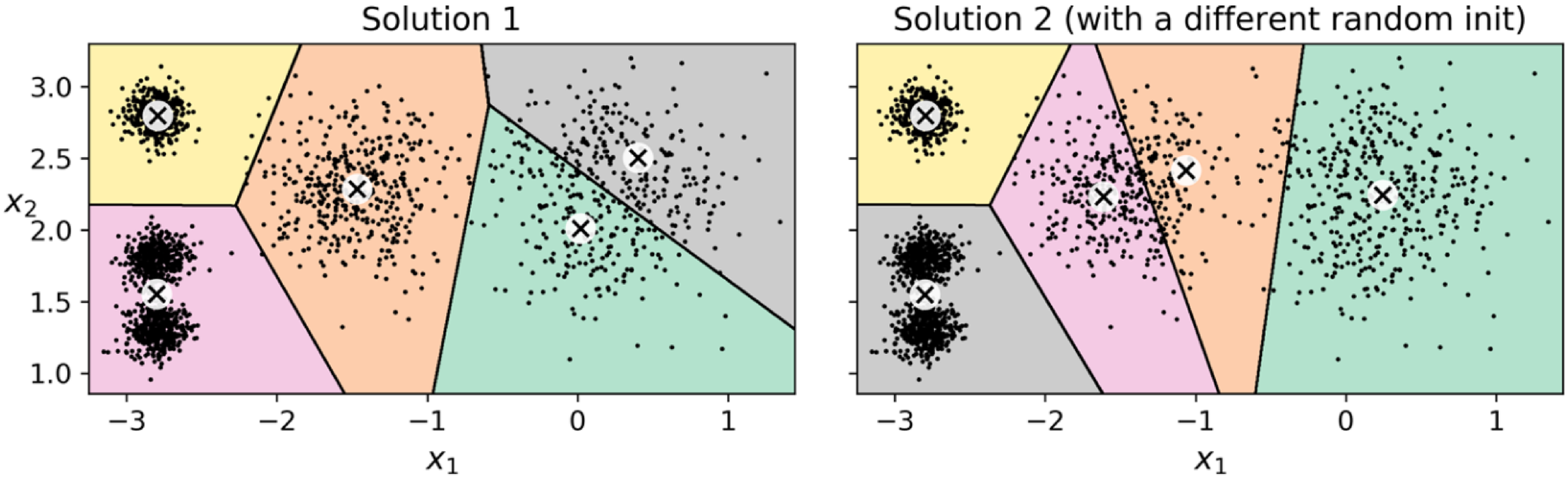


Figure 6: Suboptimal solutions due to unlucky centroid initializations.

5.2 K-means clustering

- let's look at a few ways we can mitigate this risk by improving the centroid initialization
- if we happen to know approximately where the centroids should be (e.g., if we ran another clustering algorithm earlier), then we can initialize the algorithm to start from these centroids
- another solution is to run the algorithm multiple times with different random initializations and keep the best solution
- but how exactly do we know which solution is the best?

5.2 K-means clustering

- we can use a performance metric: the metric is called the model's *inertia*, which is the mean squared distance between each instance and its closest centroid
- it is roughly equal to 223.3 for the model on the left in Figure 6, 237.5 for the model on the right in Figure 6, and 211.6 for the model in Figure 4
- the K-means algorithm will be run r times and the model with the lowest inertia will be kept
- in this example, the model in Figure 4 will be selected (unless we are very unlucky with the r consecutive random initializations)

5.2 K-means clustering

- an important improvement to the K-means algorithm, *K-means++*, was proposed
- it introduces a smarter initialization step that tends to select centroids that are distant from one another, and this improvement makes the K-means algorithm much less likely to converge to a suboptimal solution
- the additional computation required for the smarter initialization step is well worth it, because it makes it possible to drastically reduce the number of times the algorithm needs to be run to find the optimal solution
- here is the K-means++ initialization algorithm:
  1. Take one centroid $\boldsymbol{c}^{(1)}$, chosen uniformly at random from the dataset.
  2. Take a new centroid $\boldsymbol{c}^{(i)}$, choosing an instance $\boldsymbol{x}^{(i)}$ with probability

$$\frac{D(\boldsymbol{x}^{(i)})^2}{\sum_{j=1}^{n} D(\boldsymbol{x}^{(j)})^2},$$

     where $D(\boldsymbol{x}^{(i)})$ is the distance between the instance $\boldsymbol{x}^{(i)}$ and the closest centroid that was already chosen. This probability distribution ensures that instances farther away from already chosen centroids are much more likely to be selected as centroids.
  3. Repeat the previous step until all K centroids have been chosen.

5.2 K-means clustering

- another important improvement to the K-means algorithm, which considerably accelerates the algorithm by avoiding many unnecessary distance calculations, was proposed
- this is achieved by exploiting the triangle inequality (i.e., that a straight line is always the shortest distance between two points) and by keeping track of lower and upper bounds for distances between instances and centroids

5.2 *K*-means clustering

- yet another important variant of the *K*-means algorithm was proposed
- instead of using the full dataset at each iteration, the algorithm is capable of using mini-batches, moving the centroids just slightly at each iteration
- this speeds up the algorithm typically by a factor of three or four, and makes it possible to cluster huge datasets that do not fit in memory
- this is often called *mini-batch K-means*
- although the mini-batch *K*-means algorithm is much faster than the regular *K*-means algorithm, its inertia is generally slightly worse, especially as the number of clusters increases

5.2 K-means clustering

- we can see this in Figure 7: the plot on the left compares the inertias of mini-batch K-means and regular K-means models trained on the previous dataset using various numbers of clusters K
- the difference between the two curves remains fairly constant, but this difference becomes more and more significant as K increases, since the inertia becomes smaller and smaller
- in the plot on the right, we can see that mini-batch K-means is much faster than regular K-means, and this difference increases with K

5.2 K-means clustering

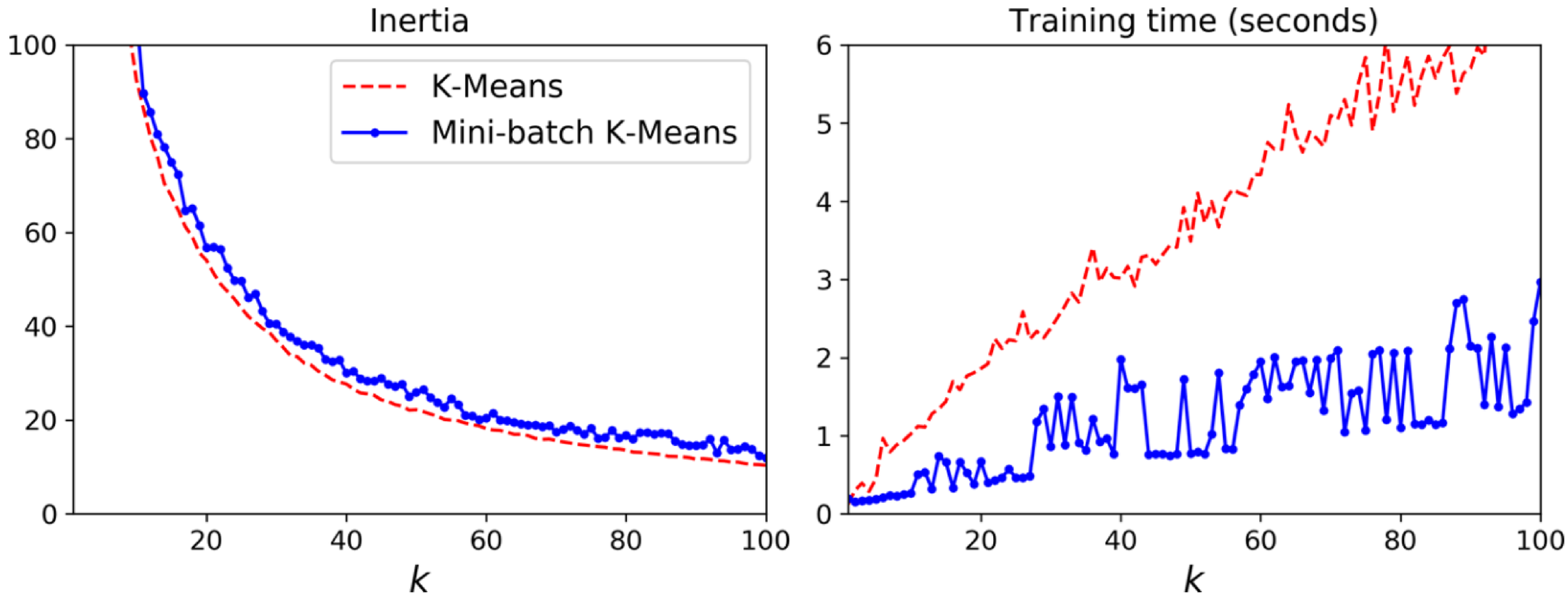


Figure 7: Mini-batch K-means has a higher inertia than K-means (left), but it is much faster (right), especially as K increases.

5.2 K-means clustering

- so far, we have set the number of clusters K to 5, because it was obvious by looking at the data that this was the correct number of clusters
- but, in general, it will not be so easy to know how to set K, and the result might be quite bad if we set it to the wrong value
- as we can see in Figure 8, setting K to 3 or 8 results in fairly bad models

5.2 K-means clustering

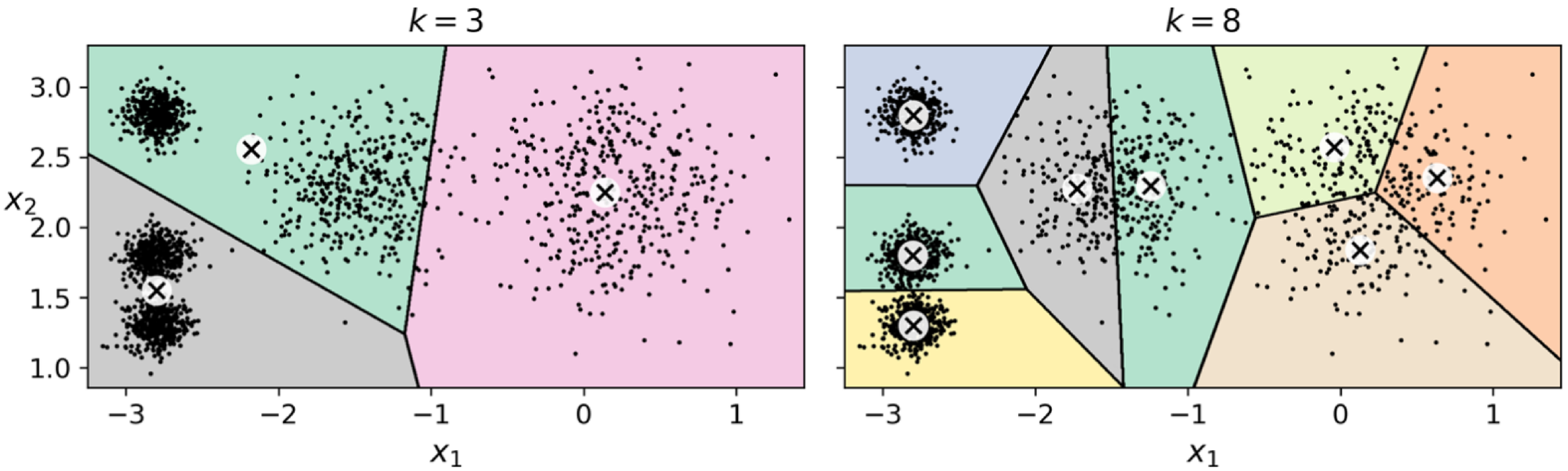


Figure 8: Bad choices for the number of clusters: when K is too small, separate clusters get merged (left), and when K is too large, some clusters get chopped into multiple pieces (right).

5.2 K-means clustering

- we might be thinking that we could just pick the model with the lowest inertia, right?
- unfortunately, it is not that simple
- the inertia for $K = 3$ is 653.2, which is much higher than for $K = 5$ (which was 211.6); but, with $K = 8$, the inertia is just 119.1
- the inertia is not a good performance metric when trying to choose K, because it keeps getting lower as we increase K
- indeed, the more clusters there are, the closer each instance will be to its closest centroid, and therefore the lower the inertia will be
- let's plot the inertia as a function of K (see Figure 9)

5.2 K-means clustering

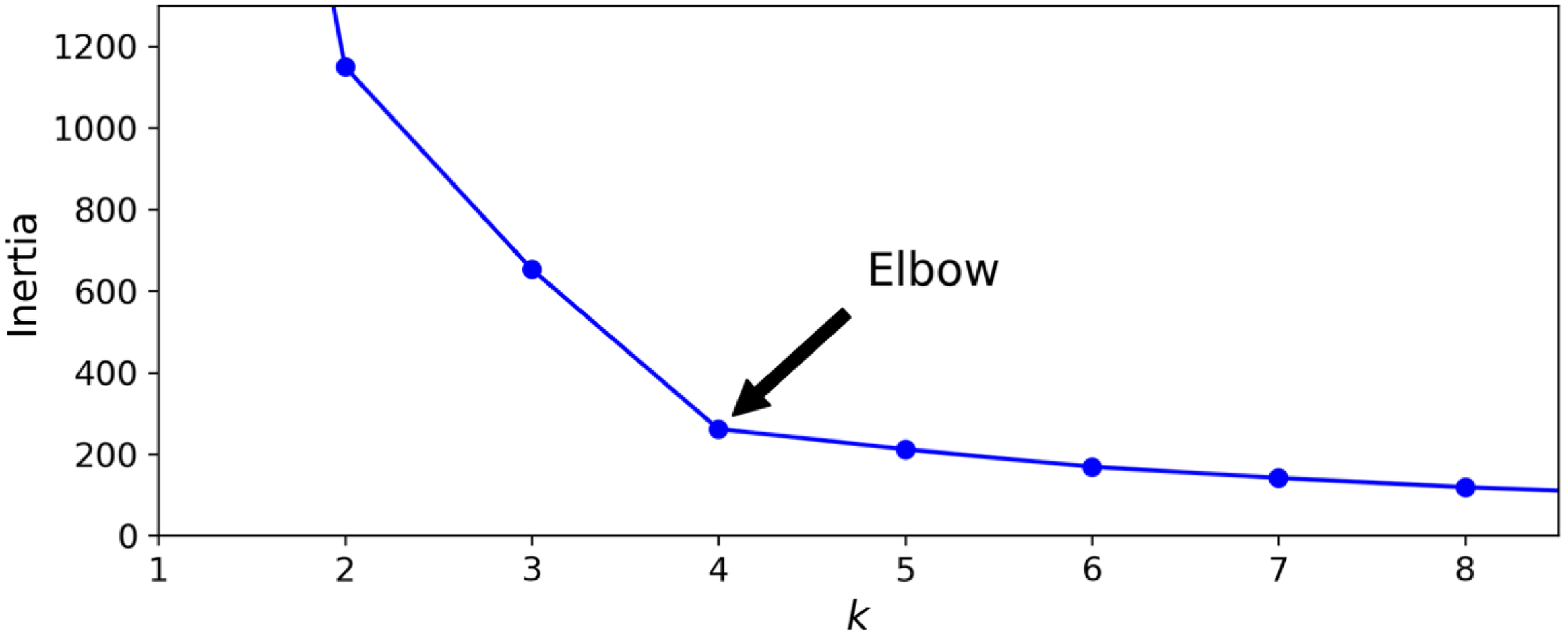


Figure 9: When plotting the inertia as a function of the number of clusters K, the curve often contains an inflexion point called the “elbow”.

5.2 K-means clustering

- as we can see, the inertia drops very quickly as we increase K up to 4, but then it decreases much more slowly as we keep increasing K
- this curve has roughly the shape of an arm, and there is an "*elbow*" at $K = 4$
- so, if we did not know better, 4 would be a good choice: any lower value would be dramatic, while any higher value would not help much, and we might just be splitting perfectly good clusters in half for no good reason
- this technique for choosing the best value for the number of clusters is rather coarse

5.2 K-means clustering

- a more precise approach (but also more computationally expensive) is to use the *silhouette score*, which is the mean *silhouette coefficient* over all the instances
- an instance's silhouette coefficient is equal to $(b - a)/\max(a, b)$, where a is the mean distance to the other instances in the same cluster (i.e., the mean intra-cluster distance) and b is the mean nearest-cluster distance (i.e., the mean distance to the instances of the next closest cluster, defined as the one that minimizes b, excluding the instance's own cluster)
- the silhouette coefficient can vary between -1 and $+1$
- a coefficient close to $+1$ means that the instance is well inside its own cluster and far from other clusters, while a coefficient close to 0 means that it is close to a cluster boundary, and finally a coefficient close to -1 means that the instance may have been assigned to the wrong cluster
- let's compare the silhouette scores for different numbers of clusters (see Figure 10)

5.2 K-means clustering

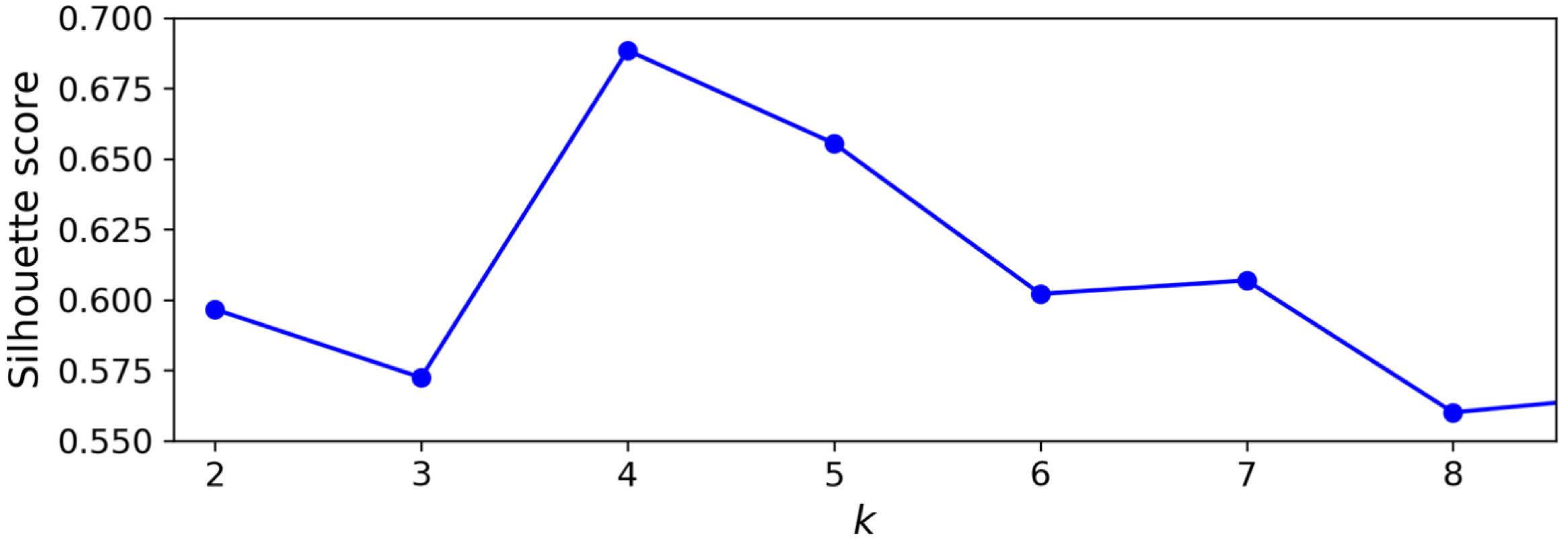


Figure 10: Selecting the number of clusters K using the silhouette score.

5.2 K-means clustering

- as we can see, this visualization is much richer than the previous one: although it confirms that $K = 4$ is a very good choice, it also underlines the fact that $K = 5$ is quite good as well, and much better than $K = 6$ or 7; this was not visible when comparing inertias
- an even more informative visualization is obtained when we plot every instance's silhouette coefficient, sorted by the cluster they are assigned to and by the value of the coefficient; this is called a *silhouette diagram* (see Figure 11)
- each diagram contains one knife shape per cluster
- the shape's height indicates the number of instances the cluster contains, and its width represents the sorted silhouette coefficients of the instances in the cluster (wider is better)

5.2 K-means clustering

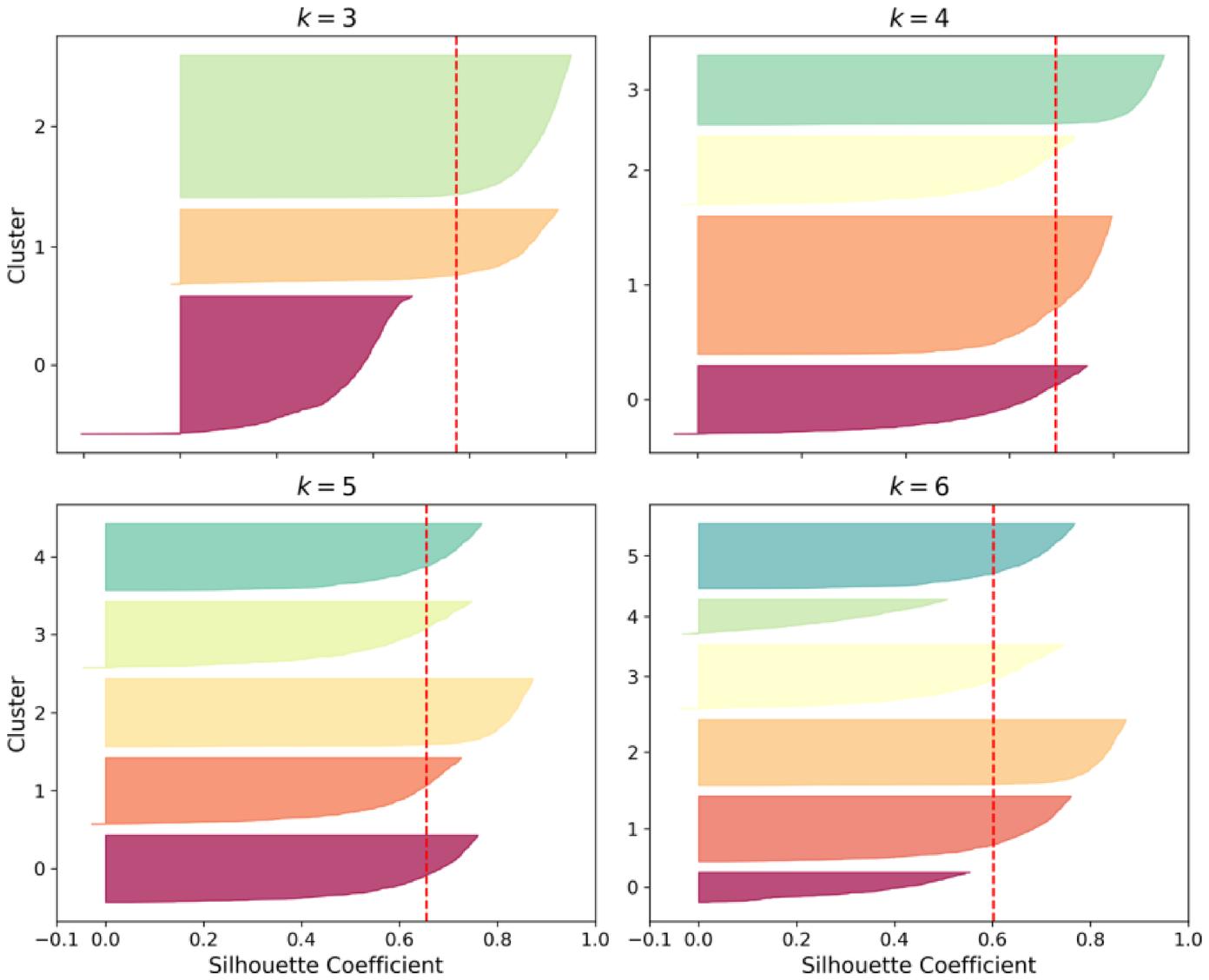


Figure 11: Analyzing the silhouette diagrams for various values of K.

5.2 K-means clustering

- the vertical dashed lines represent the silhouette score for each number of clusters
- when most of the instances in a cluster have a lower coefficient than this score (i.e., if many of the instances stop short of the dashed line, ending to the left of it), then the cluster is rather bad, since this means its instances are much too close to other clusters
- we can see that when $K = 3$ and when $K = 6$, we get bad clusters
- but when $K = 4$ or $K = 5$, the clusters look pretty good: most instances extend beyond the dashed line, to the right and closer to 1.0
- when $K = 4$, the cluster at index 1 (the third from the top) is rather big
- when $K = 5$, all clusters have similar sizes
- so, even though the overall silhouette score from $K = 4$ is slightly greater than for $K = 5$, it seems like a good idea to use $K = 5$ to get clusters of similar sizes

5.2 K-means clustering

- despite its many merits, most notably being fast and scalable, K-means is not perfect
- as we saw, it is necessary to run the algorithm several times to avoid suboptimal solutions, plus we need to specify the number of clusters, which can be quite a hassle
- moreover, K-means does not behave very well when the clusters have varying sizes, different densities, or nonspherical shapes
- for example, Figure 12 shows how K-means clusters a dataset containing three ellipsoidal clusters of different dimensions, densities, and orientations

5.2 K-means clustering

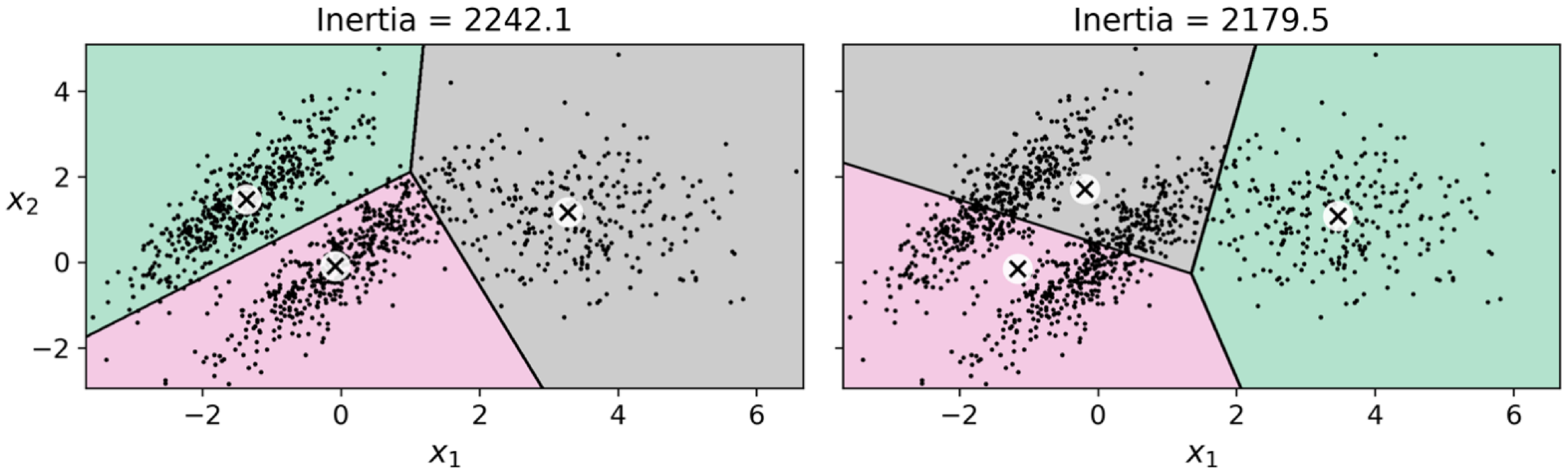


Figure 12: K-means fails to cluster these ellipsoidal blobs properly.

5.2 *K*-means clustering

- as we can see, neither of these solutions is any good
- the solution on the left is better, but it still chops off 25% of the middle cluster and assigns it to the cluster on the right
- the solution on the right is just terrible, even though its inertia is lower
- so, depending on the data, different clustering algorithms may perform better
- it is important to *scale* the input features before we run *K*-means, or the clusters may be very stretched and *K*-means will perform poorly
- scaling the features does not guarantee that all the clusters will be nice and spherical, but it generally improves things

5.3 Hierarchical clustering

- one potential disadvantage of K-means clustering is that it requires us to pre-specify the number of clusters K
- *hierarchical clustering* is an alternative approach which does not require that we commit to a particular choice of K
- hierarchical clustering has an added advantage over K-means clustering in that it results in an attractive tree-based representation of the instances, called a *dendrogram*
- there are two main approaches to hierarchical clustering: bottom-up or *agglomerative clustering*, and top-down or *divisive clustering*
- both methods take as input a dissimilarity matrix between the objects

5.3 Hierarchical clustering

- in the bottom-up approach, the most similar groups are merged at each step
- in the top-down approach, groups are split using various different criteria
- note that agglomerative and divisive clustering are both just heuristics, which do not optimize any well-defined objective function
- thus, it is hard to assess the quality of the clustering they produce, in any formal sense
- furthermore, they will always produce a clustering of the input data, even if the data has no structure at all (e.g., it is random noise)

5.3 Hierarchical clustering

- we will just discuss agglomerative clustering
- the input to the algorithm is an $n \times n$ *dissimilarity matrix* or *distance matrix* $\boldsymbol{D}$, and the output is a tree structure in which groups i and j with small dissimilarity $d_{i,j}$ are grouped together in a hierarchical fashion
- a dissimilarity matrix $\boldsymbol{D}$ is a matrix where $d_{i,i} = 0$ and $d_{i,j} \geq 0$ is a measure of "distance" between objects i and j
- subjectively judged dissimilarities are rarely distances in the strict sense, since the triangle inequality, $d_{i,j} \leq d_{i,k} + d_{j,k}$, often does not hold
- some algorithms require $\boldsymbol{D}$ to be a true distance matrix, but many do not

5.3 Hierarchical clustering

- for example, consider the set of 5 inputs points in Figure 13(a), $\boldsymbol{x}^{(i)} \in \mathbb{R}^2$
- we will use *city block distance* between the points to define the dissimilarity, i.e.:

$$d_{i,j} = \sum_{k=1}^{2} |x_k^{(i)} - x_k^{(j)}|.$$

- we start with a tree with n leaves, each corresponding to a cluster with a single data point
- next, we compute the pair of points that are closest, and merge them

5.3 Hierarchical clustering

- we see that $(1, 3)$ and $(4, 5)$ are both distance 1 apart, so they get merged first
- we then measure the dissimilarity between the sets $\{1, 3\}$, $\{4, 5\}$, and $\{2\}$ using some measure (details below), and group them, and repeat
- the result is a binary tree known as a *dendogram*, as shown in Figure 13(b)
- by cutting this tree at different heights, we can induce a different number of (nested) clusters

5.3 Hierarchical clustering

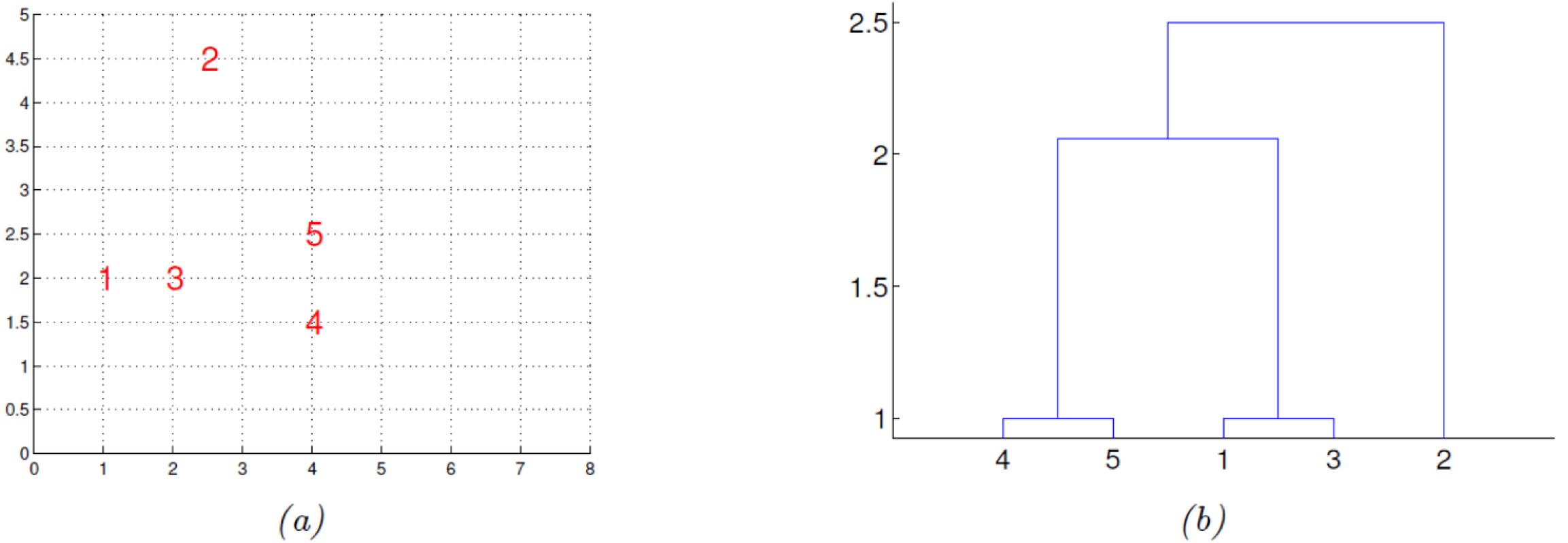


Figure 13: (a) An example of single link clustering using city block distance. Pairs $(1, 3)$ and $(4, 5)$ are both distance 1 apart, so get merged first. (b) The resulting dendrogram.

5.3 Hierarchical clustering

- agglomerative clustering starts with n groups, each initially containing one object, and then at each step it merges the two most similar groups until there is a single group, containing all the data
- since picking the two most similar clusters to merge takes $\mathcal{O}(n^2)$ time, and there are $\mathcal{O}(n)$ steps in the algorithm, the total running time is $\mathcal{O}(n^3)$
- however, by using a priority queue, this can be reduced to $\mathcal{O}(n^2 \log n)$

5.3 Hierarchical clustering

- the algorithm is given in pseudocode as:

1: *initialize* clusters as singletons: **for** $i \leftarrow 1$ **to** n **do** $C_i \leftarrow \{i\}$
2: *initialize* set of clusters available for merging: $S \leftarrow \{1, \ldots, n\}$
3: **repeat**
4: pick 2 most similar clusters to merge: $(j, k) \leftarrow \text{argmin}_{j,k \in S} d_{j,k}$
5: create new cluster $C_\ell \leftarrow C_j \cup C_k$
6: mark j and k as unavailable: $S \leftarrow S \backslash \{j, k\}$
7: **if** $C_\ell \neq \{1, \ldots, n\}$ **then**
8: mark ℓ as available: $S \leftarrow S \cup \{\ell\}$
9: **end if**
10: **for each** $i \in S$ **do**
11: update dissimilarity matrix: $d(i, \ell)$
12: **end for**
13: **until** *no more clusters are available for merging*

5.3 Hierarchical clustering

- there are actually three variants of agglomerative clustering, depending on how we define the dissimilarity between groups of objects

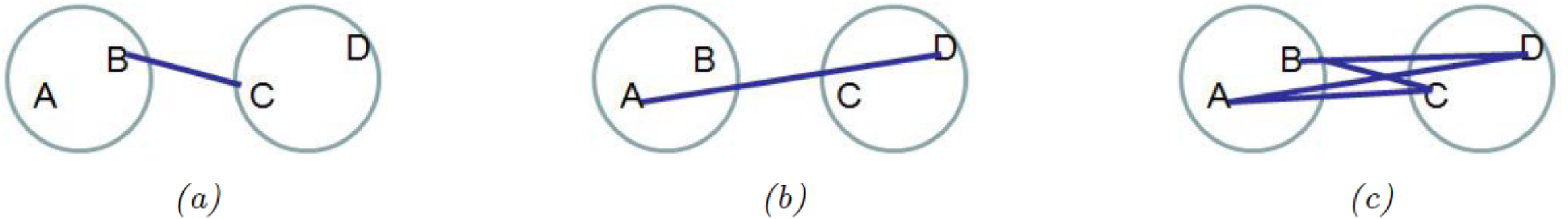


Figure 14: Illustration of (a) Single linkage. (b) Complete linkage. (c) Average linkage.

- in *single link clustering*, also called *nearest neighbor clustering*, the distance between two groups G and H is defined as the distance between the two closest members of each group (see Figure 14(a)):

$$d_{SL}(G, H) = \min_{i \in G, i' \in H} d_{i,i'}.$$

5.3 Hierarchical clustering

- the tree built using single link clustering is a minimum spanning tree of the data, which is a tree that connects all the objects in a way that minimizes the sum of the edge weights (distances)
- to see this, note that, when we merge two clusters, we connect together the two closest members of the clusters; this adds an edge between the corresponding nodes, and this is guaranteed to be the “lightest weight” edge joining these two clusters
- and once two clusters have been merged, they will never be considered again, so we cannot create cycles
- as a consequence of this, we can actually implement single link clustering in $\mathcal{O}(n^2)$ time, whereas the other variants take $\mathcal{O}(n^3)$ time

5.3 Hierarchical clustering

- in *complete link clustering*, also called *furthest neighbor clustering*, the distance between two groups is defined as the distance between the two most distant pairs (see Figure 14(b)):

$$d_{CL}(G, H) = \max_{i \in G, i' \in H} d_{i,i'}.$$

- single linkage only requires that a single pair of objects be close for the two groups to be considered close together, regardless of the similarity of the other members of the group
- thus, clusters can be formed that violate the *compactness* property, which says that all the observations within a group should be similar to each other
- in particular, if we define the *diameter* of a group as the largest dissimilarity of its members, $d_G = \max_{i \in G, i' \in G} d_{i,i'}$, then we can see that single linkage can produce clusters with large diameters
- complete linkage represents the opposite extreme: two groups are considered close only if all of the observations in their union are relatively similar
- this will tend to produce clusterings with small diameter, i.e., compact clusters

5.3 Hierarchical clustering

- in practice, the preferred method is *average link clustering*, which measures the average distance between all pairs (see Figure 14(c)):

$$d_{AL}(G, H) = \frac{1}{n_G n_H} \sum_{i \in G} \sum_{i' \in H} d_{i,i'},$$

  where n_G and n_H are the number of elements in groups G and H
- average link clustering represents a compromise between single and complete link clustering
- it tends to produce relatively compact clusters that are relatively far apart
- however, since it involves averaging of the $d_{i,i'}$'s, any change to the measurement scale can change the result
- in contrast, single linkage and complete linkage are invariant to monotonic transformations of $d_{i,i'}$, since they leave the relative ordering the same

5.4 DBSCAN

- another very useful clustering algorithm is *DBSCAN* (which stands for “density-based spatial clustering of applications with noise”)
- the main benefits of DBSCAN are that it does not require the user to set the number of clusters *a priori*, it can capture clusters of complex shapes, and it can identify points that are not part of any cluster
- DBSCAN is somewhat slower than agglomerative clustering and K-means, but still scales to relatively large datasets
- DBSCAN works by identifying points that are in “crowded” regions of the feature space, where many data points are close together
- these regions are referred to as *dense* regions in feature space
- the idea behind DBSCAN is that clusters form dense regions of data, separated by regions that are relatively empty

5.4 DBSCAN

- points that are within a dense region are called *core samples* (or core points), and they are defined as follows
- there are two parameters in DBSCAN: m, the minimum number of samples, and ε, the maximum distance between samples in the same cluster
- if there are at least m many data points within a distance of ε to a given data point, that data point is classified as a core sample
- core samples that are closer to each other than the distance ε are put into the same cluster by DBSCAN

5.4 DBSCAN

- the algorithm works by picking an arbitrary point to start with
- it then finds all points with distance ε or less from that point
- if there are less than m points within distance ε of the starting point, this point is labeled as *noise*, meaning that it doesn't belong to any cluster
- if there are more than m points within a distance of ε, the point is labeled a core sample and assigned a new cluster label

5.4 DBSCAN

- then, all neighbors (within ε) of the point are visited
- if they have not been assigned a cluster yet, they are assigned the new cluster label that was just created
- if they are core samples, their neighbors are visited in turn, and so on
- the cluster grows until there are no more core samples within distance ε of the cluster
- then another point that hasn't yet been visited is picked, and the same procedure is repeated

5.4 DBSCAN

- in the end, there are three kinds of points: core points, points that are within distance ε of core points (called *boundary points*), and noise
- when the DBSCAN algorithm is run on a particular dataset multiple times, the clustering of the core points is always the same, and the same points will always be labeled as noise
- however, a boundary point might be neighbor to core samples of more than one cluster
- therefore, the cluster membership of boundary points depends on the order in which points are visited
- usually, there are only few boundary points, and this slight dependence on the order of points is not important

5.4 DBSCAN

- increasing ε means that more points will be included in a cluster
- this makes clusters grow, but might also lead to multiple clusters joining into one
- increasing *m* means that fewer points will be core points, and more points will be labeled as noise
- the parameter ε is somewhat more important, as it determines what it means for points to be “close”
- setting ε to be very small will mean that no points are core samples, and may lead to all points being labeled as noise
- setting ε to be very large will result in all points forming a single cluster

5.4 DBSCAN

- the *m* setting mostly determines whether points in less dense regions will be labeled as noise or as their own clusters
- if we decrease *m*, anything that would have been a cluster with less than *m* many samples will now be labeled as noise
- *m* therefore determines the minimum cluster size
- while DBSCAN doesn't require setting the number of clusters explicitly, setting ε implicitly controls how many clusters will be found
- finding a good setting for ε is sometimes easier after *scaling* the data, as using scaling will ensure that all features have similar ranges

5.4 DBSCAN

- this algorithm works well if all the clusters are dense enough and if they are well separated by low-density regions
- let's test it on the `two_moons` dataset
- the clustering given by DBSCAN is represented in the left-hand plot of Figure 15
- as we can see, it identified quite a lot of noise points, plus seven different clusters
- fortunately, if we widen each instance's neighborhood by increasing ε to 0.2, we get the clustering on the right, which looks perfect

5.4 DBSCAN

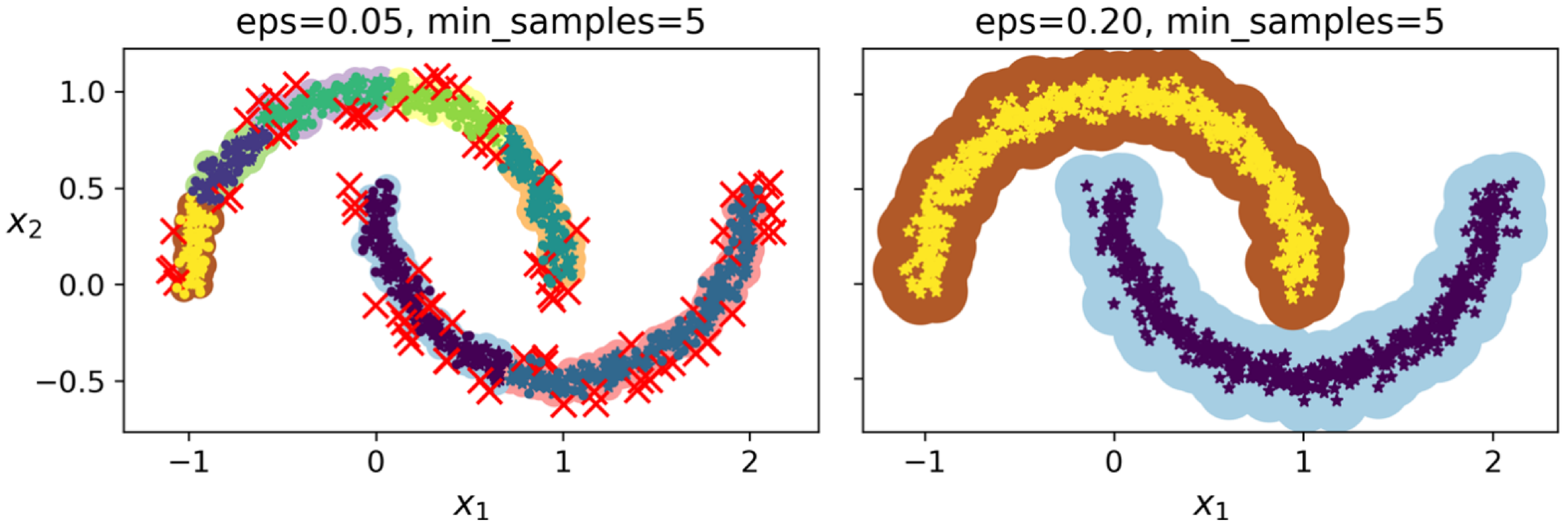


Figure 15: DBSCAN clustering using two different neighborhood radiuses.

5.4 DBSCAN

- in short, DBSCAN is a very simple yet powerful algorithm capable of identifying any number of clusters of any shape
- it is robust to outliers, and it has just two hyperparameters (ε and m)
- if the density varies significantly across the clusters, however, it can be impossible for it to capture all the clusters properly
- its computational complexity is roughly $\mathcal{O}(n \log n)$, making it pretty close to linear with regard to the number of instances, but its implementation can require up to $\mathcal{O}(n^2)$ memory, if ε is large

5.4 DBSCAN

- we looked at three clustering algorithms: K-means, agglomerative clustering, and DBSCAN
- all three have a way of controlling the granularity of clustering
- K-means and agglomerative clustering allow us to specify the number of desired clusters, while DBSCAN lets us define proximity using the ε parameter, which indirectly influences cluster size
- all three methods can be used on large, real-world datasets, are relatively easy to understand, and allow for clustering into many clusters
- each of the algorithms has somewhat different strengths

5.4 DBSCAN

- K-means allows for a characterization of the clusters using the cluster means
- it can also be viewed as a decomposition method, where each data point is represented by its cluster center
- agglomerative clustering can provide a whole hierarchy of possible partitions of the data, which can be easily inspected via dendrograms
- DBSCAN allows for the detection of “noise points” that are not assigned any cluster, and it can help automatically determine the number of clusters
- in contrast to the other two methods, it allows for complex cluster shapes, as we saw in the `two_moons` example
- DBSCAN sometimes produces clusters of very differing size, which can be a strength or a weakness

Exercises

Exercise 5

Consider a dataset containing six one-dimensional points: $\{2, 4, 7, 8, 12, 14\}$. After three iterations of hierarchical clustering using Euclidean distance between points, we get the 3 clusters: $C_1 = \{2, 4\}$, $C_2 = \{7, 8\}$, and $C_3 = \{12, 14\}$. What is the distance between clusters C_1 and C_2 using single linkage? What is the distance between clusters C_1 and C_2 using complete linkage?

Solution:

- in *single link clustering*, also called *nearest neighbor clustering*, the distance between two groups G and H is defined as the distance between the two closest members of each group:

$$d_{SL}(G, H) = \min_{i \in G, i' \in H} d_{i,i'}.$$

- in *complete link clustering*, also called *furthest neighbor clustering*, the distance between two groups is defined as the distance between the two most distant pairs:

$$d_{CL}(G, H) = \max_{i \in G, i' \in H} d_{i,i'}.$$

Exercises

- for single link clustering, we have that:

$$\begin{aligned} d_{SL}(C_1, C_2) &= \min_{i \in C_1, i' \in C_2} d_{i,i'} \\ &= \min\{|2-7|, |2-8|, |4-7|, |4-8|\} \\ &= \min\{5, 6, 3, 4\} \\ &= 3 \end{aligned}$$

- for complete link clustering, we have that:

$$\begin{aligned} d_{CL}(C_1, C_2) &= \max_{i \in C_1, i' \in C_2} d_{i,i'} \\ &= \max\{|2-7|, |2-8|, |4-7|, |4-8|\} \\ &= \max\{5, 6, 3, 4\} \\ &= 6 \end{aligned}$$

Vă mulţumesc!